

Teoría de Algoritmos

(75.29 / 95.06) Curso Ing. Podberezski

Trabajo Práctico Nro. 1

21 de Abril de 2025

Grupo: **BigO**

Apellido y Nombre	Padrón	e-mail
Czerwiak, Juan Joaquin	109.640	jczerwiak@fi.uba.ar
Huzan, Hugo Javier	67.910	hhuzan@fi.uba.ar
Rivero, Julián Agustín	112.454	jarivero@fi.uba.ar
Rojo, Santiago	110.700	srojo@fi.uba.ar

Parte 1: Solitario... en grupo.....	3
¿Todas las propuestas fueron greedy?.....	3
Complejidad.....	3
Mejores y Peores.....	3
Fueron todas óptimas?.....	4
Demostración.....	4
Parte 2: Los puentes que se cruzan.....	5
Solución propuesta.....	5
Ecuación de recurrencia y Complejidad.....	5
Procesamiento de entrada.....	5
Algoritmo de conteo de inversiones.....	6
Ecuación de recurrencia.....	6
Complejidad total.....	6
Pseudocódigo.....	6
Ejemplo Paso a Paso.....	9
Detalle del conteo de inversiones para el ejemplo:.....	10
Programa en Python.....	11
Complejidad de la Implementación.....	11
Parte 3: Maximizando los puentes.....	12
Solución propuesta.....	12
Definición de Subproblema.....	12
Relación de Recurrencia.....	13
Pseudocódigo.....	13
Explique por qué su propuesta funciona.....	16
Complejidad.....	16
Espacial.....	17
Temporal.....	17
Ejemplo paso a paso.....	18
Detalle del algoritmo LIS para el ejemplo.....	18
Reconstrucción:.....	19
Resultado Final.....	19
Implementación.....	20
Complejidad Propuesta Vs. Programada.....	20
¿Único resultado posible?.....	20
Posible modificación:.....	20
Ejemplo:.....	21
Referencias.....	22

Parte 1: Solitario... en grupo

Considerar el problema “Jugando al solitario” en el trabajo práctico 0. Cada participante del grupo elaboró una propuesta algorítmica al problema. Deben compararla y responder las consignas justificando.

¿Todas las propuestas fueron greedy? ^{1 2}

Todos los algoritmos propuestos fueron de tipo greedy, ya que las cinco soluciones toman una estrategia greedy, que consiste en dividir el problema en subproblemas, donde se aplica una solución basada únicamente en la información disponible en ese momento (la carta actual y las pilas ya creadas), con la expectativa de que estas decisiones lleven a una solución global óptima.

Cada nueva carta se coloca en la primera pila cuyo tope sea mayor que la carta evaluada (es decir, la menor posible que cumpla esta condición), o se crea una nueva pila si no existe tal pila.

Esta estrategia selecciona una solución local óptima sin considerar las cartas que aún no han sido reveladas.

Complejidad

Para definir la complejidad se usa la variable n para referirse a la cantidad de cartas.

Propuesta	Alumno	Com. Temporal	Comp. Espacial
1	Czerwiak	$O(n^2)$	$O(n)$
2	Huzan	$O(n \cdot \log(n))$	$O(n)$
3	Olmedo	$O(n^2)$	$O(n)$
4	Rivero	$O(n \cdot \log(n))$ ³	$O(n)$ ⁴
5	Rojo	$O(n \cdot \log(n))$	$O(n)$

Mejores y Peores

Las propuestas 2,4 y 5 tuvieron mejor complejidad temporal. $T(n) = O(n \cdot \log(n))$.

El resto (prop. 1 y 3) $T(n) = O(n^2)$

En todos los casos la complejidad espacial fue $O(n)$

¹ Greedy [Apunte](#)

² Greedy [Kleinberg_Tardos](#)

³ En el informe figura $O(n \cdot \log(p))$ con p = cant. de pilas

⁴ En el informe figura $O(p)$ con p = cant. de pilas

Fueron todas óptimas?

Si. Todos los algoritmos propuestos fueron óptimos. Aunque estén implementados de diversas maneras, los 5 algoritmos tienen la misma idea general para la solución del problema, por lo que la siguiente demostración es válida para cualquiera de ellos.

Demostración

Se demuestra la optimalidad del algoritmo por inducción:⁵

Para el **caso base** de una sola carta, el algoritmo crea una pila y coloca el único elemento disponible en ella. Es claramente óptimo, no vamos a poder usar menos de una pila si tenemos aunque sea una carta.

Suponiendo que el algoritmo es óptimo para n cartas significa que la cantidad de pilas necesarias para acomodar esas cartas es la mínima.

Ahora, considerando el caso de $n + 1$ cartas, el algoritmo colocará la carta adicional en la primera pila con el tope mayor al nuevo elemento, o en su defecto lo coloca en una nueva pila si no existe tal tope.

Si la nueva carta se coloca en una pila existente, la solución sigue siendo óptima porque el número de pilas no se modificó respecto al anterior caso, y partimos de la base de que la solución es óptima para n cartas.

Si, en cambio, colocamos la nueva carta en una pila nueva, es porque no existe una pila con el tope mayor al nuevo elemento. En este caso es obligatorio crear una pila nueva, por lo que la solución sigue siendo óptima.

Ahora que probamos que el caso de $n + 1$ cartas es óptimo, podemos repetir esta lógica para ver que el algoritmo es óptimo para cualquier cantidad de cartas.

⁵ Demostración por [Inducción](#)

Parte 2: Los puentes que se cruzan

Un río navegable separa a una ciudad en dos regiones: norte y sur. La ciudad tiene " $2n$ " barrios. Exactamente hay " n " barrios en cada margen. La municipalidad quiere mejorar la conectividad construyendo puentes. Cada puente debe unir un barrio del norte con uno del sur. Las propuestas de construcción son realizadas por las comunas de cada barrio. Cada barrio solo puede ser parte de una única propuesta. El comité de evaluación recibe las propuestas y debe determinar la factibilidad de la construcción del conjunto de puentes. Decimos que no es factible si existen puentes que se cruzan. En caso de no ser posible debe informar cuantos cruces de puentes existen.

Solución propuesta

La solución propuesta se basa en el algoritmo para **contar inversiones** visto en la clase "División y Conquista" de la materia (*). El cual a su vez se basa en el algoritmo "Merge-Sort" (*).⁶ Dado que el algoritmo visto sirve para contar inversiones respecto al orden "natural", y en nuestro caso debemos contarlas entre dos listas desordenadas, se ordena respecto a una para luego trabajar con la otra en ese orden natural. Por simplicidad y eficiencia se transforman los nombres de los barrios a números de acuerdo a la entrada de *barrios.txt*.

Ecuación de recurrencia y Complejidad

Para la medición de complejidad se va a usar n para referirse a la cantidad de barrios en cada margen como se especifica en el enunciado.

Procesamiento de entrada

Procesar la entrada resulta en una complejidad, tanto **espacial** como **temporal**, de $O(n)$ en cada caso.

- *barrios.txt*: Contiene la información de los $2n$ barrios. Procesarlo requiere una complejidad temporal $O(2n)$ para recorrer todos los barrios y almacenarlos en dos diccionarios (norte y sur), lo que también implica una complejidad espacial $O(2n)$.
- *propuesta.txt*: Contiene las n propuestas de puentes. Se procesa en tiempo $O(n)$ y se construye un arreglo de tamaño n donde el índice representa el barrio del sur según el orden de *barrios.txt* y el valor almacenado representa el barrio del norte conectado, resultando en una complejidad espacial $O(n)$.

⁶ *División y conquista*

Algoritmo de conteo de inversiones

Ordenar y contar inversiones se realiza en complejidad temporal $O(n \log n)$, consumiendo un espacio adicional $O(n)$.

- Cada problema de n elementos se subdivide en 2 subproblemas de $n/2$ elementos.
- Se resuelve cada subproblema recursivamente.
- El caso base ocurre cuando el subarreglo tiene menos de un elemento y retorna cero inversiones en tiempo constante.
- Durante la combinación, se cuentan las inversiones cruzadas entre las dos mitades en tiempo $O(n)$.

Ecuación de recurrencia

La **ecuación de recurrencia** queda $T(n) = 2T(n/2) + O(n)$, con $A = 2$ (subproblemas), $B = 2$ (reducción de tamaño) y $f(n) = n$ (costo de combinación).

Cómo $f(n) = \Theta(n^{\log_b(a)}) = \Theta(n^{\log_2(2)}) = \Theta(n^1) = \Theta(n)$, se aplica el segundo caso del teorema Maestro. Por lo tanto:

$$T(n) = O(n \cdot \log(n))$$

Complejidad total

- Complejidad temporal: $T(n) = O(n)$ [procesamiento de entrada] + $O(n \cdot \log(n))$ [conteo de inversiones] + $O(1)$ [operaciones constantes] = $O(n \cdot \log(n))$.
- Complejidad espacial: $S(n) = O(2n)$ [estructuras para almacenar barrios] + $O(n)$ [arreglo para contar inversiones] = $O(n)$.

Pseudocódigo

Contar Cruces

```
diccionarios = leer_barrios()
LISTA = leer_propuesta(diccionarios)
CANTIDAD_INVERSIONES = Ordenar_y_Contar_Inversiones(LISTA)
Imprimir CANTIDAD_INVERSIONES
```

Ordenar y Contar Inversiones

```
Ordenar_Y_Contar_Inversiones(LISTA):  
    Si la LISTA contiene un 'unico elemento // caso base  
        Retornar 0  
    Si no  
        Sea PARTE_IZQ la primera mitad de la LISTA  
        Sea PARTE_DER la última mitad de la LISTA  
        Ordenar_Y_Contar_Inversiones(PARTE_IZQ)  
        Ordenar_Y_Contar_Inversiones(PARTE_DER)  
        CANTIDAD_INVERSIONES = Mezclar_Y-Contar_Inversiones(PARTE_IZQ, PARTE_DER)  
        Retornar CANTIDAD_INVERSIONES
```

Mezclar y Contar Inversiones

```
Mezclar_Y_Contar_Inversiones(PARTE_IZQ, PARTE_DER):  
    Sea i=1 puntero al Parte-Izq  
    Sea j=1 puntero al PARTE_DER  
    Sea TEMPORAL un array vacío  
    Cantidad_Inversiones = 0  
  
    Repetir mientras (i < tamaño de Izquierda) y (j < tamaño de Derecha)  
        si PARTE_IZQ[i] < PARTE_DER[j]  
            agregar PARTE_IZQ[i] a TEMPORAL  
            incrementar i  
        Si no  
            Agregar PARTE_DER[j] a TEMPORAL  
            Incrementar j  
            Cantidad_Inversiones += tamaño de PARTE_IZQ - i  
  
    Si quedaron elementos de PARTE_IZQ sin agregar // i < tamaño de PARTE_IZQ  
        Agregar los elementos restantes de PARTE_IZQ a TEMPORAL  
    Si quedaron elementos de PARTE_DER sin agregar // j < tamaño de Parte-Der  
        Agregar los elementos restantes de PARTE_DER a TEMPORAL  
    Retornar Cantidad_Inversiones
```

Lectura de Barrios

```
leer_barrios():  
    Sean DICCIONARIO_NORTE y DICCIONARIO_SUR diccionarios vacíos  
  
    índice = 0  
    Por cada línea leída hasta llegar a una en blanco:  
        Leer BARRIO  
        Agregar {BARRIO, índice} al DICCIONARIO_NORTE  
        Incrementar índice  
  
    índice = 0  
    Por cada línea hasta el fin de archivo:  
        Leer BARRIO  
        Agregar {BARRIO, índice} a DICCIONARIO_SUR  
        Incrementar índice  
  
    Retornar diccionarios
```

Lectura de Propuesta

```
leer_propuesta(DICCIONARIO_NORTE, DICCIONARIO_SUR):  
    Sea PROPUESTA una lista de tamaño igual a la cant. de barrios por lado  
  
    Por cada línea leída del archivo de PROPUESTA:  
        leer LADO_NORTE y LADO_SUR  
        índice_norte = DICCIONARIO_NORTE[LADO_NORTE]  
        índice_sur = DICCIONARIO_SUR[LADO_SUR]  
  
        PROPUESTA[índice_norte] = índice_sur  
  
    Retornar PROPUESTA
```


Ejemplo Paso a Paso

Se ejemplifica con el ejemplo del enunciado:

Se comienza leyendo desde el primer archivo los dos conjuntos de nombres de los barrios

A medida que se va leyendo se van insertando los nombres en 2 diccionarios usando como clave el nombre de los barrios y como valor el orden en que fueron leídos (orden geográfico de los barrios).

Barrancas	0	Cienagas	0
San Pedro	1	Don Corleone	1
El hurón	2	Barrio Este	2
Palo seco	3	Portuario	3
Puente Viejo	4	Torre verde	4

Se crea un array **propuesta** de longitud **n** (cantidad de barrios por lado)

Se va leyendo el segundo archivo

Por cada línea leída se hace **propuesta[índice(norte)] = índice(sur)**

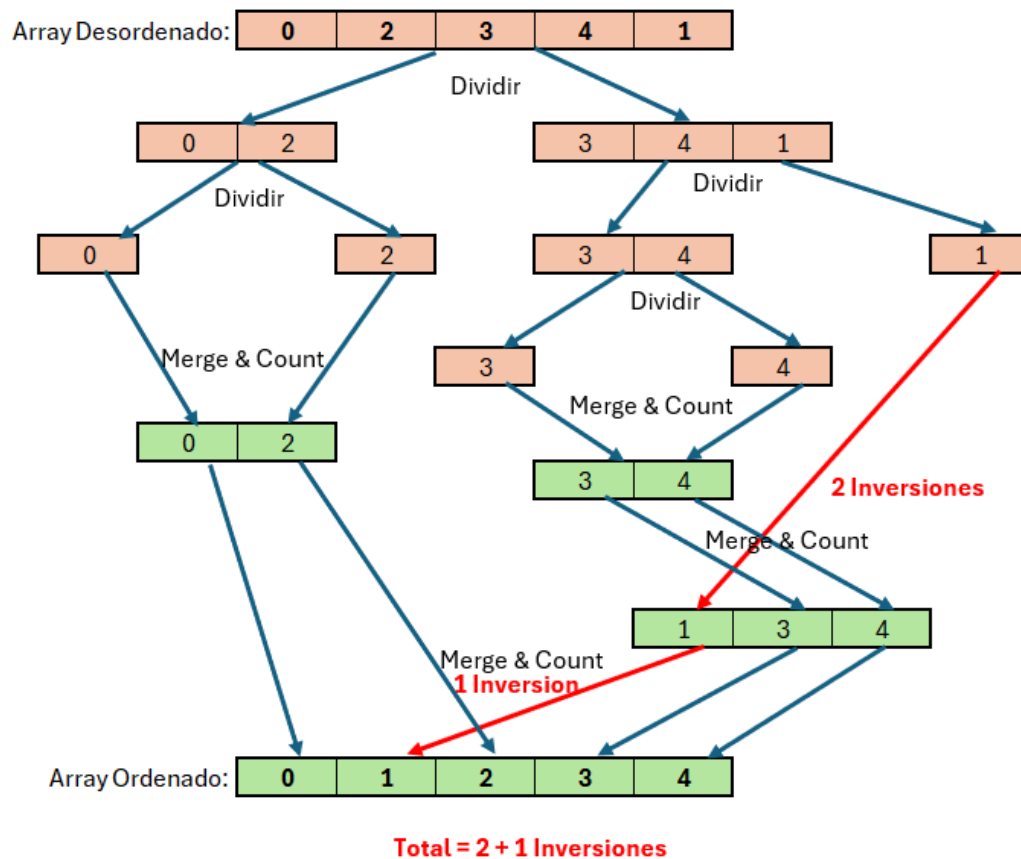
(no es necesario guardar los nombres, solo el array generado)

Puente Viejo	4	1	Don Corleone	=>	propuesta[4] = 1
Barrancas	0	0	Cienagas	=>	propuesta[0] = 0
El hurón	2	3	Portuario	=>	propuesta[2] = 3
San Pedro	1	2	Barrio Este	=>	propuesta[1] = 2
Palo seco	3	4	Torre verde	=>	propuesta[3] = 4

Finalmente se aplica el “**algoritmo de conteo de inversiones**” sobre el array **propuesta = [0,2,3,4,1]**

Detalle del conteo de inversiones para el ejemplo:

propuesta: **[0, 2, 3, 4, 1]**



Resultado final

Inversiones en **[0,2]** = 0

Inversiones en **[3,4,1]** = 2

Inversiones cruzadas = 1

Total = 0 + 2 + 1 = 3 inversiones

La cantidad de inversiones resultantes (**3**) corresponde a los **3 cruces** de puentes existentes en la propuesta.

Programa en Python

Requisitos:

- Python 3

Ejecución:

```
python3 puentes_dq.py archivoBarrios.txt archivoPropuesta
```

Complejidad de la Implementación

En la implementación en **python** propuesta se mantiene la complejidad temporal y espacial teórica calculada.

Si bien la implementación utiliza el **slicing** de **python** para completar la parte final de la copia del array en `mezclar_y_contar_inversiones()`, la complejidad temporal de esta operación ($T(n) = O(n)$) fue contemplada en el análisis del pseudocódigo.

Parte 3: Maximizando los puentes

Continuamos con la propuesta de construcción de puentes. Nos informan que debemos determinar qué puentes construir dado la propuesta evitando los cruces y maximizando la cantidad de puentes a construir.

Solución propuesta

Se propuso maximizar la cantidad de puentes sin cruces al reducir el problema y utilizar el algoritmo *Longest increasing subsequences* (LIS) en los índices de los barrios del Sur ordenados previamente por los índices de los barrios del Norte en el orden dado por *barrios.txt*.^{7 8 9}

Para lograr lo dicho se siguieron los siguiente procedimientos:

1. Leer el primer archivo (barrios) y armar diccionarios que permitan el pasaje de nombre a orden y de orden a nombre.
2. Leer el segundo archivo (propuesta) e ir armando un arreglo con las posiciones del extremo sur de los puentes respecto del lado norte. Adicionalmente se arma un índice al extremo norte para poder reconstruir los nombres de los extremos de cada puente.
3. Aplicar el algoritmo LIS al array de las posiciones del extremo sur.
4. Imprimir la subsecuencia más larga de puentes sin cruces al interpretar por medio de los diccionarios el resultado obtenido por el paso anterior.

Definición de Subproblema¹⁰

Subproblema: ¿Cuál es la longitud de la subsecuencia creciente más larga que termina en la posición i del arreglo?

$dp_{(i)}$ = longitud de la subsecuencia creciente más larga que termina en la posición i del arreglo.

Para el caso base: $i = 1 \Rightarrow dp_{(i)} = 1$

⁷ Referirse a la sección "[Explique por qué su propuesta funciona](#)" para una mayor justificación.

⁸ Algoritmo [Longest Increasing Subsequence](#)

⁹ Cap 6 [Kleinberg Tardos](#)

¹⁰ Apunte [Programación Dinámica](#)

Relación de Recurrencia

Sea $V = [a_1, a_2, \dots, a_n]$ una lista de n elementos.

Definimos $dp_{(i)}$ la longitud de la LIS que termina en la posición i (es decir, que incluye a a_i).

$$dp_{(i)} = 1 + \max(dp_{(j)}) \forall j < i / a_j < a_i$$
$$\text{si no existe } j < i / a_j < a_i \Rightarrow dp_{(i)} = 1$$

Pseudocódigo

Maximizar Cantidad de Puentes

```
listas y diccionarios = leer_barrios()
PROPUESTA y índices = leer_propuesta(listas)

resultado = LIS(PROPUESTA)

Imprimir longitud de resultado

Para cada elemento de resultado
    imprimir LISTA_NORTE[ind_a_norte[elemento]] y LISTA_SUR[elemento]
```

Algoritmo LIS

```
LIS(LISTA):
    Sea n la longitud de la LISTA
    Sea OPT un array de longitud igual a la lista con 1s en todas las posiciones
    Sea PREV un array de longitud igual a la lista con -1 en todas las posiciones
    MAX = 0
    Para i desde 0 hasta n:
        Para j desde 0 hasta i:
            Si LISTA[j] < LISTA[i]:
                OPT[i] = max( OPT[i], 1 + OPT[j] )
                Si se eligió 1 + OPT[j]:
                    PREV[i] = j
        Si OPT[i] > OPT[MAX]:
            MAX = i

    // reconstrucción de la secuencia
    Sea SUB_MAX un array vacío
    idx_fin = MAX
    Mientras idx_fin <> -1:
        insertar LISTA[idx_fin] en SUB_MAX
        idx_fin = PREV[idx_fin]
    retornar SUB_MAX
```

Este algoritmo tiene Complejidad Temporal $T_{(n)} = O(n^2)$.

Es posible mejorarlo usando una estructura auxiliar que guarda el menor posible valor final para una subsecuencia de cada longitud.

Esta estructura se mantiene siempre ordenada, por lo que es posible buscar en la misma en forma binaria sin necesidad de recorrerla en forma lineal.

De esta forma se logra una Complejidad Temporal $T_{(n)} = O(n \cdot \log(n))$.

LIS Optimizado [$T_{(n)} = O(n \cdot \log(n))$]

```
LIS(LISTA):
    Sea n la longitud de la lista
    Sea FIN_MIN un array vacío
    Sea PREV un array de longitud n con -1 en todas las posiciones
    Sea INDICES un array de longitud n, vacío

    Para i desde 0 hasta n - 1:
        Usar búsqueda binaria del índice j en FIN_MIN tal que
                                LISTA[FIN_MIN[j]] >= LISTA[i]

        Si se encuentra:
            FIN_MIN[j] = i
        Si No:
            Añadir i al final de FIN_MIN

        Si j > 0:
            PREV[i] = INDICES[j - 1]

        INDICES[j] = i

    // reconstrucción de la secuencia
    Inicializar INDEX al valor del último elemento de FIN_MIN
    Sea MAX_SUB una lista vacía

    Mientras INDEX <> -1 :
        Insertar lista[index] en MAX_SUB
        INDEX = PREV[INDEX]

    Invertir MAX_SUB

    Retornar MAX_SUB
```

Lectura de Barrios:

```
leer_barrios():
    Sean DICCIONARIO_NORTE y DICCIONARIO_SUR diccionarios vacíos
    Sean LISTA_NORTE y LISTA_SUR dos listas vacías

    índice = 0
    Por cada línea leída hasta llegar a una en blanco:
        Leer BARRIO
        Agregar BARRIO a la LISTA_NORTE
        Agregar {BARRIO, índice} al diccionario_norte
        Incrementar índice

    índice = 0
    Por cada línea hasta el fin de archivo:
        Leer BARRIO
        Agregar BARRIO a la LISTA_SUR
        Agregar {BARRIO, índice} a diccionario_sur
        Incrementar índice

    Retornar diccionarios y listas
```

Lectura de Propuesta

```
leer_propuesta(DICCIONARIO_NORTE, DICCIONARIO_SUR):
    Sea PROPUESTA una lista de tamaño igual a la cant. de barrios por lado
    Sea INDICES_PROPUESTA_NORTE una lista de tamaño igual a la cant. de barrios por lado

    Por cada línea leída del archivo de propuesta:
        leer LADO_NORTE y LADO_SUR
        índice_norte = DICCIONARIO_NORTE[LADO_NORTE]
        índice_sur = DICCIONARIO_SUR[LADO_SUR]

        PROPUESTA[indice_norte] = índice_sur
        INDICES_PROPUESTA_NORTE[ind_sur] = índice_norte

    Retornar PROPUESTA e INDICES_PROPUESTA_NORTE
```

Explique por qué su propuesta funciona

Para que dos puentes no se crucen, sus extremos deben conservar el orden:

- Si un puente $A = (\text{Norte}_1, \text{Sur}_1)$ y otro puente $B = (\text{Norte}_2, \text{Sur}_2)$, entonces si $\text{Norte}_1 < \text{Norte}_2$, debe cumplirse $\text{Sur}_1 < \text{Sur}_2$ para que no se crucen.

Visualmente, se cruzan si el orden relativo en un lado no coincide con el orden relativo en el otro lado.

Para que ningún puente se cruce con otro, todos los pares de puentes deben cumplir el criterio anterior. Esto implica que en un set de puentes sin cruces, si los extremos norte están ordenados también tendrán que estarlo los del extremo sur:

Orden creciente en Lado Norte + Orden creciente en Lado Sur $\Rightarrow$ Puentes no se cruzan

Nuestra estrategia consiste en tomar en orden los puentes por el *lado norte*. Luego, sobre los valores correspondientes al *lado sur*, buscamos una **subsecuencia estrictamente creciente**.

Este problema se reduce exactamente al resuelto por el algoritmo **LIS**:

- Dada una secuencia $a_1, a_2, \dots, a_n$ encontrar la subsecuencia estrictamente creciente más larga.

La razón por la que funciona la técnica de LIS es que, al tomar ordenada una dimensión (lado norte), el problema de evitar intersecciones se traduce directamente en preservar el orden en la otra dimensión (lado sur).

Finalmente la **Subsecuencia Creciente Más Larga** representa el mayor subconjunto de puentes donde los extremos sur mantienen el orden, asegurando que no haya cruces.

Complejidad

Para la medición de complejidad se va a usar n para referirse a la cantidad de barrios en cada margen como se utilizó para la parte 2.

Los mecanismos de leer los archivos e imprimir el resultado son, en esencia, muy sencillos y similares a la parte 2 del informe. En estos mecanismos se utilizan estructuras de datos en los cuales se guardan y/o se recorren n elementos. Dado esto, diremos que estos mecanismos “simples” tienen una complejidad espacial y temporal total $O(n)$ y que la complejidad del programa estará dada por la suma de estos y el algoritmo LIS utilizado.

Nota: el algoritmo LIS utilizado es el algoritmo optimizado.

Espacial

Para la complejidad del algoritmo LIS se utilizaron estas estructuras:

- *INDICES*: contiene un mapeo de índice de la subsecuencia al arreglo original. Tiene, teóricamente, el mismo tamaño que la subsecuencia.
- *PREV*: contiene el índice previo del arreglo original al índice del arreglo original. Tiene longitud n .
- *FIN_MIN*: es un arreglo auxiliar que, para cada longitud i , almacena el menor valor posible que puede ser el último elemento de una subsecuencia creciente de longitud i .
- *MAX_SUB*: es la lista en la que se reconstruye el resultado. Posee la subsecuencia más larga.

Se puede apreciar que todas estas estructuras almacenan a lo sumo la misma cantidad de elementos que el arreglo pasado. Entonces, la complejidad espacial de este algoritmo es $S(n) = O(n)$

Luego, la complejidad espacial del programa es:

$$S(n) = O(n)[\text{mecanismos simples}] + O(n)[\text{algoritmo LIS}] = O(n)$$

Temporal

El algoritmo LIS recorre todo el arreglo una vez en su totalidad donde:

1. Se busca la posición del elemento en *FIN_MIN* en tiempo logarítmico, dado que el mismo está ordenado.
2. Se actualizan las estructuras de acuerdo a la posición en tiempo constante.

Entonces se reconstruye la solución. Esta reconstrucción llega un tiempo proporcional al tamaño de la subsecuencia más larga, la cual en el peor de los casos es igual a la longitud del arreglo de entrada.

Luego, la complejidad temporal del algoritmo LIS queda dada por:

$$T(n) = O(n \log n)[\text{búsqueda LIS}] + O(n)[\text{reconstrucción}] = O(n \log n)$$

La complejidad temporal del programa queda así:

$$T(n) = O(n)[\text{mecanismos simples}] + O(n \log n)[\text{algoritmo LIS}] = O(n \log n)$$

Ejemplo paso a paso

En forma similar a la parte 2 se lee el *archivo de barrios*, generando los diccionarios y también 2 listas que permiten conocer los nombres de los barrios partiendo de su posición geográfica:

dic_norte =>		dic_sur =>	
Barrancas	0	Cienagas	0
San Pedro	1	Don Corleone	1
El hurón	2	Barrio Este	2
Palo seco	3	Portuario	3
Puente Viejo	4	Torre verde	4
<= llista_norte		<= lista_sur	

Leyendo el *archivo de propuesta* se genera el vector **propuesta** junto a un **array de índices** a los barrios del norte para poder reconstruir los nombres de los puentes que es posible construir.

Puente Viejo	4	1	Don Corleone	=>	propuesta[4] = 1	=>	ind_a_norte[1] = 4
Barrancas	0	0	Cienagas	=>	propuesta[0] = 0	=>	ind_a_norte[0] = 0
El hurón	2	3	Portuario	=>	propuesta[2] = 3	=>	ind_a_norte[3] = 2
San Pedro	1	2	Barrio Este	=>	propuesta[1] = 2	=>	ind_a_norte[2] = 1
Palo seco	3	4	Torre verde	=>	propuesta[3] = 4	=>	ind_a_norte[4] = 3

Se aplica el “**algoritmo LIS**” sobre el array **propuesta**

propuesta = **[0,2,3,4,1]**

resultado: **[0,2,3,4]**

Detalle del algoritmo LIS para el ejemplo

LISTA = **[0,2,3,4,1]**

i	num = LISTA[i]	posición de num en FIN_MIN	FIN_MIN	INDICES	PREV
0	0	0	[0]	[0]	[-1,-1,-1,-1]
1	2	1	[0,2]	[0,1]	[-1,0,-1,-1,-1]
2	3	2	[0,2,3]	[0,1,2]	[-1,0,1,-1,-1]
3	4	3	[0,2,3,4]	[0,1,2,3]	[-1,0,1,2,-1]
4	1	1	[0,1,3,4]	[0,4,2,3]	[-1,0,1,2,0]

Reconstrucción:

Luego reconstruimos desde **INDICES[3] = 3** hacia atrás:

- $FIN_MIN[3] = 4 \rightarrow MAX_SUB = [4]$ y $prev = 2$
- $FIN_MIN[2] = 3 \rightarrow MAX_SUB = [4,3]$ y $prev = 1$
- $FIN_MIN[1] = 2 \rightarrow MAX_SUB = [4,3,2]$ y $prev = 0$
- $FIN_MIN[0] = 0 \rightarrow MAX_SUB = [4,3,2,0]$ y $prev = -1$ (fin)

Finalmente **LIS** = invertir(MAX_SUB) = invertir($[4,3,2,0]$) = **[0,2,3,4]**

Resultado Final

LIS = [0,2,3,4]

Cantidad de puentes = Longitud de LIS = **4**

Para cada **elemento** del resultado del LIS corresponde un puente cuyos nombres son:

$lista_norte[indice_norte[elemento]] \Rightarrow lista_sur[elemento]$:

Puentes a Construir:

LIS	Barrio Norte		Barrio Sur
0	$indice_norte[0] = 0$	$lista_norte[0] =$ Barrancas	$lista_sur[0] =$ Cienagas
2	$indice_norte[2] = 1$	$lista_norte[1] =$ San Pedro	$lista_sur[2] =$ Barrio Este
3	$indice_norte[3] = 2$	$lista_norte[2] =$ El hurón	$lista_sur[3] =$ Portuario
4	$indice_norte[4] = 3$	$lista_norte[3] =$ Palo seco	$lista_sur[4] =$ Torre verde

Implementación

Requisitos:

- Python 3

Ejecución:

```
python3 puentes_pd.py archivoBarrios.txt archivoPropuesta
```

Complejidad Propuesta Vs. Programada

La implementación del algoritmo en **Python** tiene la misma complejidad que la solución teórica optimizada propuesta: $T(n) = O(n \cdot \log(n))$.

Las tres funciones implementadas [`secuencia_creciente_mas_larga()`, `leer_barrios()` y `leer_propuesta()`] siguen el mismo algoritmo descrito en el pseudocódigo y no se realizan copias adicionales de estructuras de tamaño variable pues los pasajes de parámetros se hacen por referencia de objeto mutable.

¿Único resultado posible?

De un contraejemplo si no lo es o una demostración si lo es. En caso de no serlo, ¿podrían modificar rápidamente para obtener una solución diferente?

Dado una propuesta de puentes, la cantidad posible de puentes a construir será siempre la misma.

Sin embargo la selección que se haga puede variar. Por ejemplo en un escenario mínimo de sólo dos puentes, los cuales se cruzan; se podría elegir cualquiera de estos.

Posible modificación:

Una posible modificación es cambiar el índice del número por el que se empieza a reconstruir el resultado de la secuencia incremental más larga.

En vez de que se empiece a reconstruir por el índice del último número en la subsecuencia FIN_MIN, se propone empezar por el índice del último agregado a la misma.

Esta modificación cambiaría el arreglo final al, en vez de devolver la secuencia incremental más larga con el número máximo más pequeño; devolver la secuencia incremental más larga con el número máximo más grande.

Esta modificación conlleva calcular el índice del número con el cuál empezar cada vez que se agregue un elemento al FIN_MIN, en vez de calcularlo al finalizar el bucle for.

Acá se muestra el pseudo código modificado:

```
LIS (LISTA) :  
...  
    INDICE_MAX = -1  
  
    Para i desde 0 hasta n - 1:  
        Usar búsqueda binaria del índice j en FIN_MIN tal que  
                                LISTA[FIN_MIN[j]] >= LISTA[i]  
  
        Si se encuentra:  
            FIN_MIN[j] = i  
            INDICE_MAX = i  
  
        Si No:  
            Añadir i al final de FIN_MIN  
  
        Si j > 0:  
            PREV[i] = INDICES[j - 1]  
  
        INDICES[j] = i  
  
    // reconstrucción de la secuencia  
Inicializar INDEX al valor del último elemento de FIN_MIN  
    Inicializar INDEX con INDICE_MAX  
...  
  
    Retornar MAX_SUB
```

Ejemplo:

Dado dos puentes que se reducen a una secuencia de números **[1, 0]**, el algoritmo LIS utilizado devuelve **[0]** mientras que el modificado devuelve **[1]**.

Referencias

1. Podberezski. V.. "Apunte TDA - Metodología Greedy." Google Drive, 2025.
<https://docs.google.com/document/d/1NO1DQJmM-Ym1CZzsSiok03wLChCOV6AreIMt0X-tXfM/edit#heading=h.nrnw03t7conb>
2. Podberezski. V.. "Diapositivas TDA - Greedy Interval Scheduling: Inducción" Google Drive, 2025.
<https://drive.google.com/file/d/11wRU9Fmk2N6puqq9STyKelB2VuQjf4fx/edit>
3. Podberezski. V.. "Apunte TDA -. División y Conquista" Google Drive, 2025.
<https://docs.google.com/document/d/1wOfctjnnNPV3FBCJSV3L9PhjZQazowkqUQpvs-wlU0A/edit?tab=t.0#heading=h.nrnw03t7conb>
4. Podberezski. V.. "Apunte TDA -.Programación Dinámica" Google Drive, 2025.
<https://docs.google.com/document/d/1XUA4yWLXqz7n6SWtgFRIDp6GKPU3A2ZHNSQjyLYXMZk/edit?tab=t.0#heading=h.nrnw03t7conb>
5. Algorithm Design Kleinberg-Tardos "Cap 4 - Greedy Algorithms"
Addison Wesley (2006). ISBN 0-321-29535-8
6. Algorithm Design Kleinberg-Tardos "Cap 6 - Dynamic Programming"
Addison Wesley (2006). ISBN 0-321-29535-8
7. Wikipedia "Longest Increasing Subsequence"
<https://dl.icdst.org/pdfs/files3/9ce98c127c79e548ebea18966f526ae9.pdf>